

JavaScript: Hoisting та Closures

Головна ідея: Hoisting пояснює, чому оголошення змінних і функцій ніби "піднімаються" вгору коду ще до виконання. **Closure** (замикання) -- це коли функція "запам'ятовує" змінні з того оточення, де вона була створена, і має до них доступ навіть після того, як зовнішня функція завершилась. Разом вони пояснюють багато "магічної" поведінки JS, яка інакше здається незрозумілою.

1. Hoisting -- підняття оголошень

Перед виконанням коду інтерпретатор JavaScript проходить по ньому і заздалегідь реєструє (запам'ятовує) усі оголошення змінних і функцій у поточній області видимості. Це і називається hoisting -- ніби оголошення "піднімаються" на початок блоку/файлу, хоча фізично код не переміщується.

var -- піднімається, ініціалізується undefined

```
console.log(x); // undefined -- не помилка!
var x = 5;
console.log(x); // 5

// Насправді інтерпретатор бачить це так:
var x;           // оголошення піднялось нагору
console.log(x);  // undefined
x = 5;           // присвоєння лишилось на місці
console.log(x);  // 5
```

function declaration -- піднімається повністю (з тілом)

```
sayHi(); // "Hi!" -- працює, хоча виклик ДО оголошення в коді

function sayHi() {
  console.log("Hi!");
}
```

Звичайне оголошення функції (`function name() {}`) підіймається **разом з тілом** -- тому її можна викликати в будь-якому місці функції/файлу, навіть до рядка, де вона фізично написана.

let / const -- теж піднімаються, але з "тимчасовою мертвою зоною" (TDZ)

```
console.log(y); // ReferenceError: Cannot access 'y' before initialization
let y = 5;

console.log(z); // ReferenceError
const z = 10;
```

Temporal Dead Zone (TDZ): `let` і `const` технічно теж підіймаються, але залишаються "неініціалізованими" від початку блоку до рядка з фактичним оголошенням. Спроба звернутись до змінної в цій зоні дає `ReferenceError`, а не `undefined` -- на відміну від `var`. Саме тому `let / const` вважаються безпечнішими: помилка стає видимою одразу, а не ховається за тихим `undefined`.

Функціональний вираз -- підіймається лише як змінна

```
sayBye(); // TypeError: sayBye is not a function
var sayBye = function () {
  console.log("Bye!");
};
// Піднялось лише "var sayBye;" (undefined), саме присвоєння функції -- ні
```

Спосіб оголошення	Що підіймається	Звернення до підняття
<code>var x</code>	оголошення (без значення)	<code>undefined</code>
<code>let / const</code>	оголошення (в TDZ)	<code>ReferenceError</code>
<code>function foo() {}</code>	оголошення + тіло цілком	працює як звичайний виклик
<code>const foo = function() {}</code>	тільки змінна (як <code>const / var</code>)	<code>ReferenceError</code> / <code>TypeError</code>

2. Closures -- замикання

Closure (замикання) -- це функція разом із "запам'ятованим" оточенням (лексичним контекстом), у якому вона була створена. Навіть якщо зовнішня функція вже завершила виконання, внутрішня функція все одно має доступ до її змінних.

```
function makeCounter() {
  let count = 0;           // "приватна" змінна зовнішньої функції

  return function () {     // внутрішня функція -- замикання
    count += 1;
    return count;
  };
}

const counter = makeCounter(); // makeCounter() вже завершилась...
counter(); // 1 -- ...але внутрішня функція все ще пам'ятає count
counter(); // 2
counter(); // 3

const counter2 = makeCounter(); // новий, повністю НЕЗАЛЕЖНИЙ виклик
counter2(); // 1 -- своя власна змінна count, не пов'язана з першим лічильником
```

Кожен виклик `makeCounter()` створює **нову** змінну `count` і нову функцію-замикання навколо неї. Замикання -- це не одна спільна "коробка", а окрема для кожного виклику зовнішньої функції.

Навіщо потрібні замикання

Застосування	Як допомагає closure
Приватний стан (без класів)	змінна в зовнішній функції недоступна ззовні напряму -- тільки через повернуту функцію, як у прикладі з <code>count</code> вище
Функції-фабрики	функція, яка повертає іншу функцію, "налаштовану" під конкретні параметри (наприклад, <code>makeMultiplier(3)</code> повертає функцію, що завжди множить на 3)
Колбеки, що пам'ятають контекст	колбек у <code>setTimeout</code> /обробнику подій "бачить" змінні з місця, де його створили, навіть виконуючись пізніше
Кешування результатів	зовнішня функція тримає "приховане" сховище (об'єкт/Map), до якого має доступ лише повернута функція

```
// Функція-фабрика: кожен виклик makeMultiplier "запам'ятовує" своє число
function makeMultiplier(factor) {
  return function (num) {
    return num * factor; // factor -- зі замикання, різний для кожної фабрики
  };
}

const double = makeMultiplier(2);
const triple = makeMultiplier(3);

double(5); // 10
triple(5); // 15 -- незалежно від double, бо кожен виклик має свій factor
```

Класична пастка: closures у циклі з var

```
// ✖ Проблема: усі функції "бачать" ОДНУ й ту саму змінну i
for (var i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Виведе: 4 4 4 -- бо var не має блокової області видимості,
// до моменту виконання колбеків цикл вже закінчився і i = 4

// ✔ Рішення: let створює НОВУ змінну i на кожній ітерації
for (let i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Виведе: 1 2 3 -- кожне замикання "бачить" свою окрему i
```

Це один з найпоширеніших прикладів на співбесідах: `let` має блокову область видимості, тому кожна ітерація циклу створює власну копію змінної, яку "хапає" своє замикання. `var` такої ізоляції не має.

3. Витік пам’яті (memory leak) через closures

Замикання тримає посилання на змінні зовнішньої функції **увесь час, поки саме замикання існує й досягне** (наприклад, збережене в глобальній змінній або прив'язане як обробник подій). Якщо замикання випадково "тримає" великий об'єкт, який насправді вже не потрібен, -- цей об'єкт не може бути прибраний збирачем сміття (garbage collector), і пам'ять просто накопичується.

Приклад витоку

```
function attachHandler() {
  const largeData = new Array(1_000_000).fill("leak"); // великий масив у пам'яті

  document.getElementById("btn").addEventListener("click", function () {
    console.log(largeData.length); // замикання тримає весь largeData,
  }); // хоча реально потрібна лише його довжина
}
attachHandler();
```

Навіть якщо кнопку `#btn` видалили з DOM, обробник (а разом з ним і `largeData`) залишиться в пам'яті, доки хтось явно не викличе `removeEventListener` -- бо замикання утримує посилання на весь зовнішній контекст, а не тільки на те, що реально використовується всередині.

Як уникати

- Забирай тільки потрібне. Якщо всередині колбека потрібна лише довжина масиву -- виріши це до створення замикання: `const len = largeData.length;` і використовуй `len` у колбеку, а не весь `largeData`.
- Прибирай обробники подій, коли вони більше не потрібні: `element.removeEventListener(...)`, або опція `{ once: true }` для одноразових.
- Обнуляй посилання на великі дані, коли вони вже відпрацювали: `largeData = null;` (якщо це можливо за логікою коду).
- Для довгострокових кешів використовуй `WeakMap` / `WeakSet` замість звичайного об'єкта/ `Map` -- вони не заважають збирачу сміття видаляти ключі-об'єкти, якщо на них більше ніхто не посилається.

```
// Виправлена версія -- замикання тримає лише число, не весь масив
function attachHandler() {
  const largeData = new Array(1_000_000).fill("leak");
  const length = largeData.length; // беремо тільки те, що треба

  document.getElementById("btn").addEventListener("click", function () {
    console.log(length); // замикання тримає лише "length", не largeData
  });
}
```

4. Швидкий довідник

Поняття	Суть
Hoisting	оголошення змінних/функцій реєструються на початку області видимості ще до виконання коду
<code>var</code> hoisting	підіймається оголошення, значення -- <code>undefined</code> до рядка присвоєння
<code>let</code> / <code>const</code> hoisting	підіймається, але в TDZ -- звернення до рядка оголошення дає <code>ReferenceError</code>
function declaration	підіймається повністю з тілом -- можна викликати до місця в коді
Temporal Dead Zone (TDZ)	проміжок від початку блоку до рядка оголошення <code>let</code> / <code>const</code> , де змінна існує, але недоступна
Closure (замикання)	функція + запам'ятоване оточення, у якому вона була створена

Приватний стан через closure	змінна зовнішньої функції недоступна напряму ззовні, тільки через повернуту функцію
Функція-фабрика	зовнішня функція повертає "налаштовану" функцію, що пам'ятає передані їй параметри
Closure в циклі (var vs let)	<code>var</code> -- усі колбеки бачать одну спільну змінну; <code>let</code> -- кожна ітерація має свою
Memory leak через closure	замикання утримує в пам'яті весь зовнішній контекст, навіть непотрібні великі дані, доки саме замикання досяжне
WeakMap / WeakSet	структури, що не заважають збирачу сміття очистити об'єкт-ключ, якщо на нього більше ніхто не посилається